

Module 15 - chapter 22

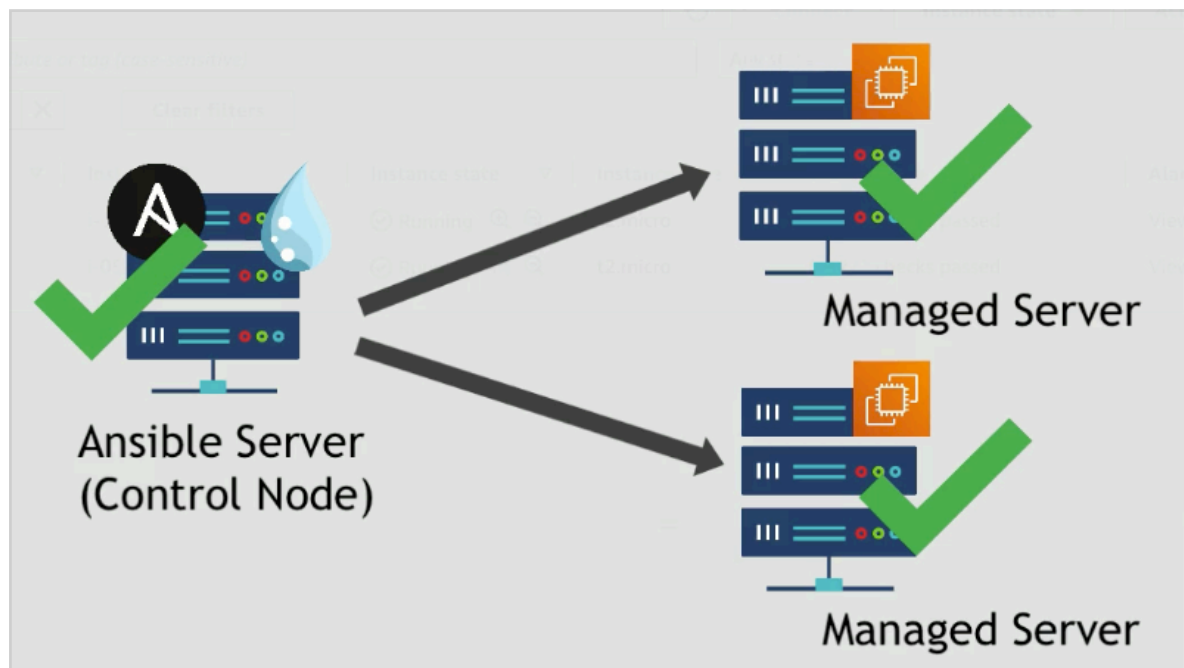
Project: Run Ansible from Jenkins Pipeline - Part 2

Starting code: https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/java-maven-app/-/tree/master?ref_type=heads

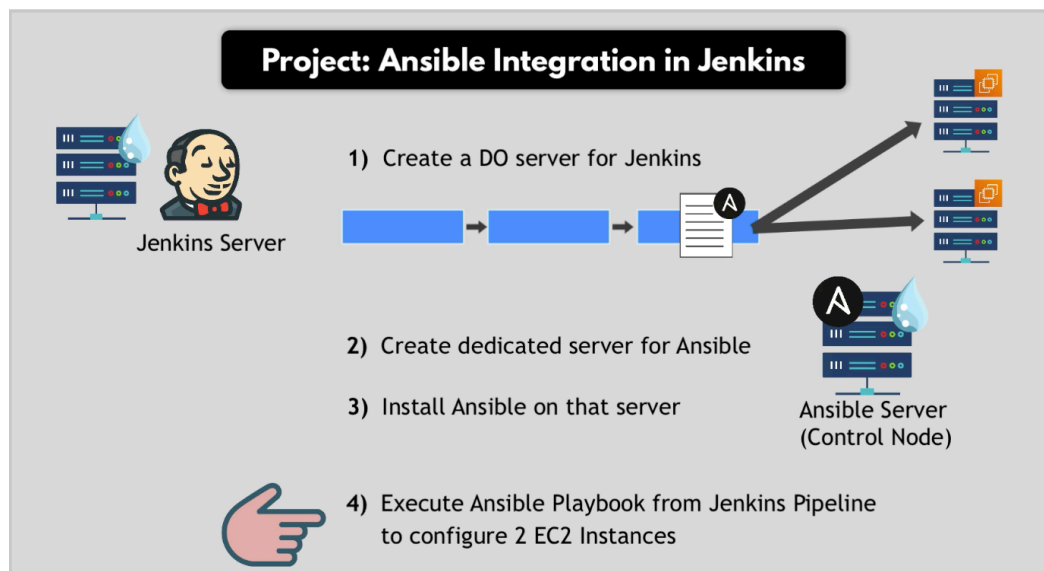
Final code: https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/java-maven-app/-/tree/feature/ansible?ref_type=heads

Jenkinsfile: Copy Files from Jenkins to Ansible Server

So far we have our Ansible control Node and two EC2 instances, ansible managed nodes:



Next



We will start by writing a Jenkinsfile that copies all the files that we need to a remote Ansible server and then executes an ansible playbook on the remote server.

And we will use our java-maven app for this

```
Ansible_chapter-22 > Jenkinsfile
Jenkinsfile
1 pipeline {
2   agent any
3   stages {
4     stage("copy files to ansible server") {
5       steps {
6         script {
7           echo "copying all necessary files to ansible control node"
8         }
9       }
10    }
11  }
12 }
13
```

Now let's see which file we need to copy to the ansible server in order to execute the playbook:

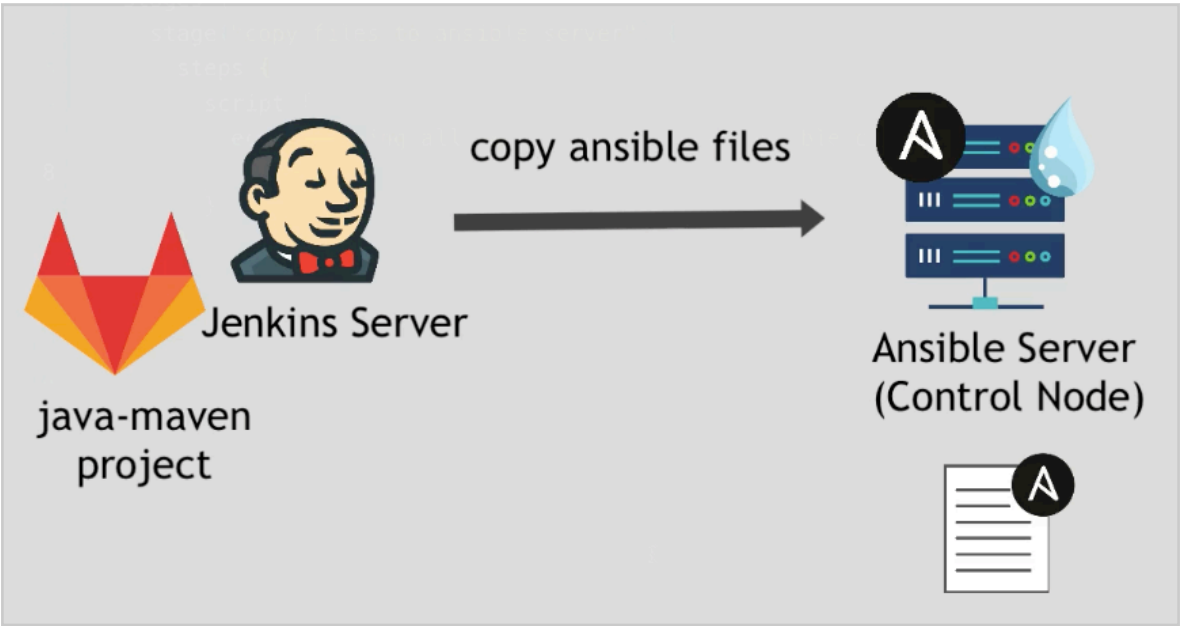
- Playbook
- Either 'hosts' file or 'dynamic inventory' file. In this demo we will use the AWS EC2 dynamic inventory file
- Ansible configuration -> ansible.cfg

```
16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
- name: Install Docker
hosts: tag_Name_prod_server
become: yes
tasks:
  - name: Make sure Docker is installed
    yum:
      name: docker
      update_cache: yes
      state: present
  - name: Ensure Docker is running
    systemd:
      name: docker
      state: started

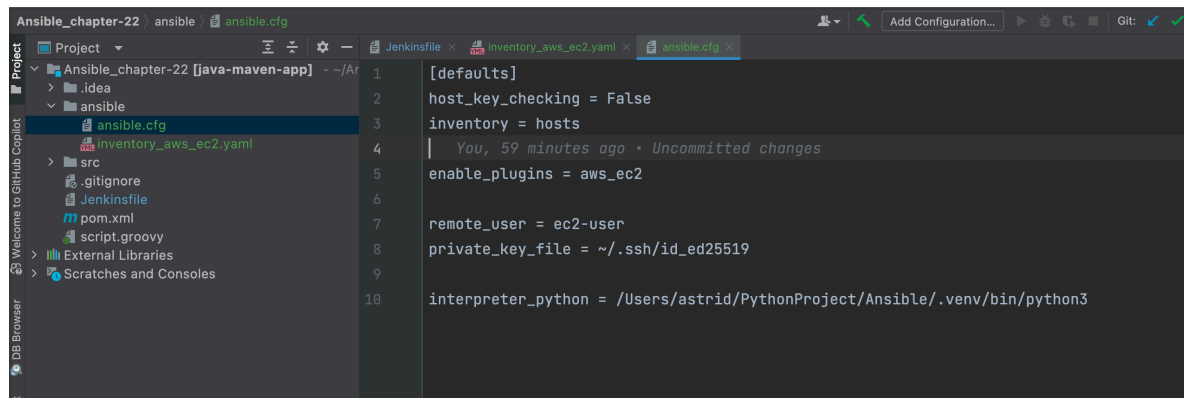
- name: Create new linux user
hosts: tag_Name_prod_server
vars_files:
  - project-vars
become: yes
tasks:
  - name: Create new linux user
    user:
      name: astrid
      groups: "{{user_groups}}"

1 2 3 4 5 6 7 8 9 10
---
plugin: aws_ec2
regions:
  - eu-west-2
#filters:
#  tag:Name: dev*
# instance-state-name: running
keyed_groups:
  - key: tags
    prefix: "tag"
```

So we need to add the above three files to our java-maven project so jenkins has access to them in order to copy them to ansible server



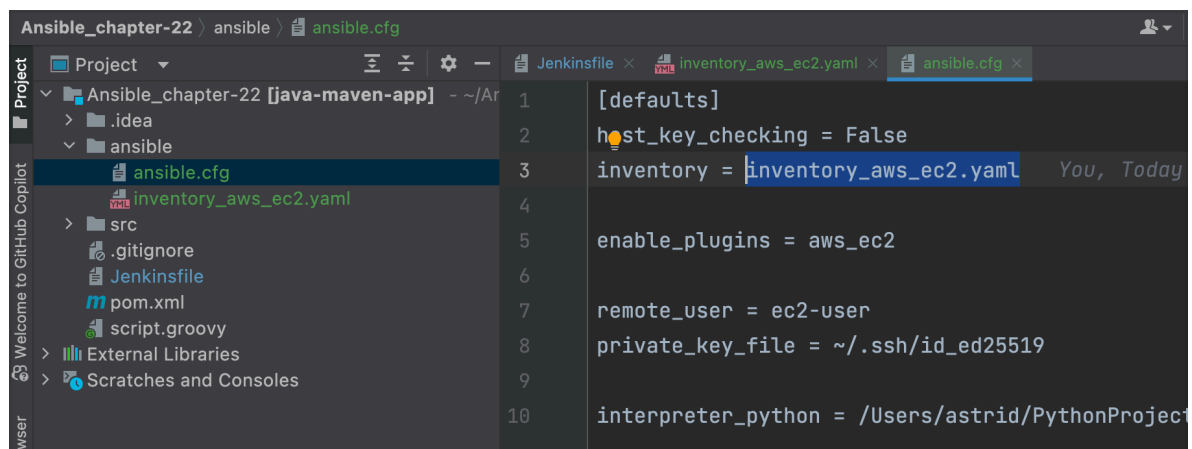
In our java-maven-app we create a new folder called 'ansible' and we copy the files inside that folder:



```
[defaults]
1 host_key_checking = False
2 inventory = hosts
3
4 | You, 59 minutes ago * Uncommitted changes
5 enable_plugins = aws_ec2
6
7 remote_user = ec2-user
8 private_key_file = ~/.ssh/id_ed25519
9
10 interpreter_python = /Users/astrid/PythonProject/Ansible/.venv/bin/python3
```

And this point we make some editions to ansible.cfg

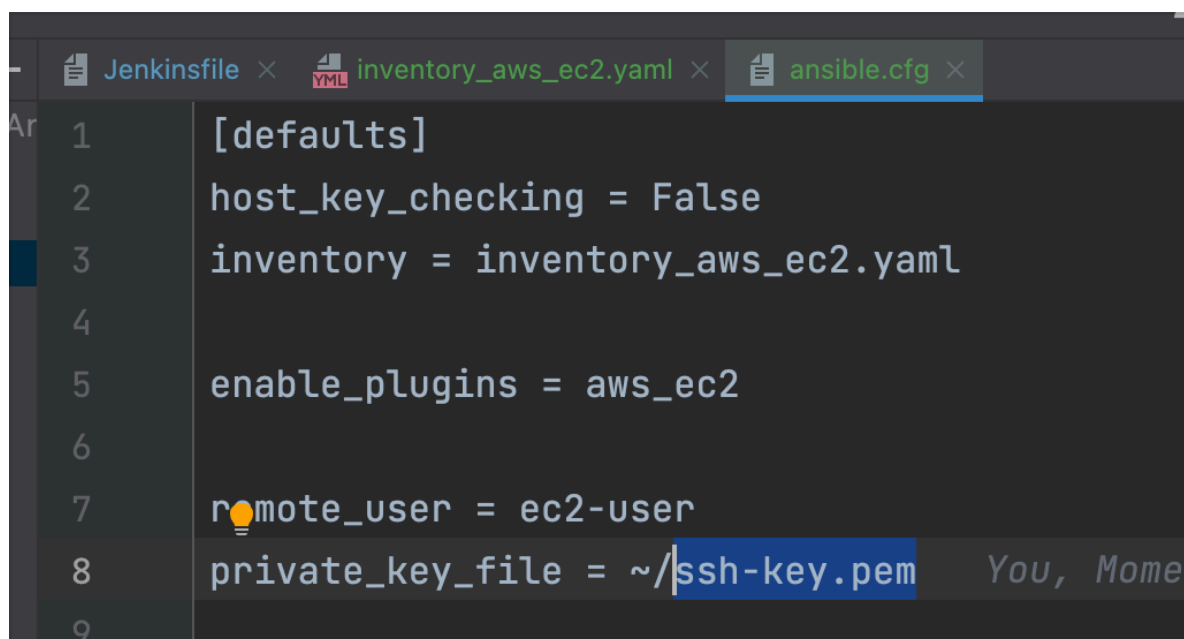
- Update the inventory reference file:



```
[defaults]
1 host_key_checking = False
2 inventory = inventory_aws_ec2.yaml
3
4
5 enable_plugins = aws_ec2
6
7 remote_user = ec2-user
8 private_key_file = ~/.ssh/id_ed25519
9
10 interpreter_python = /Users/astrid/PythonProject
```

Then update the private key as it is the one created by AWS that we downloaded. We will copy the pem file to ansible server from jenkins.

We can decide how we call the pem file in ansible server like 'ssh-key.pem'

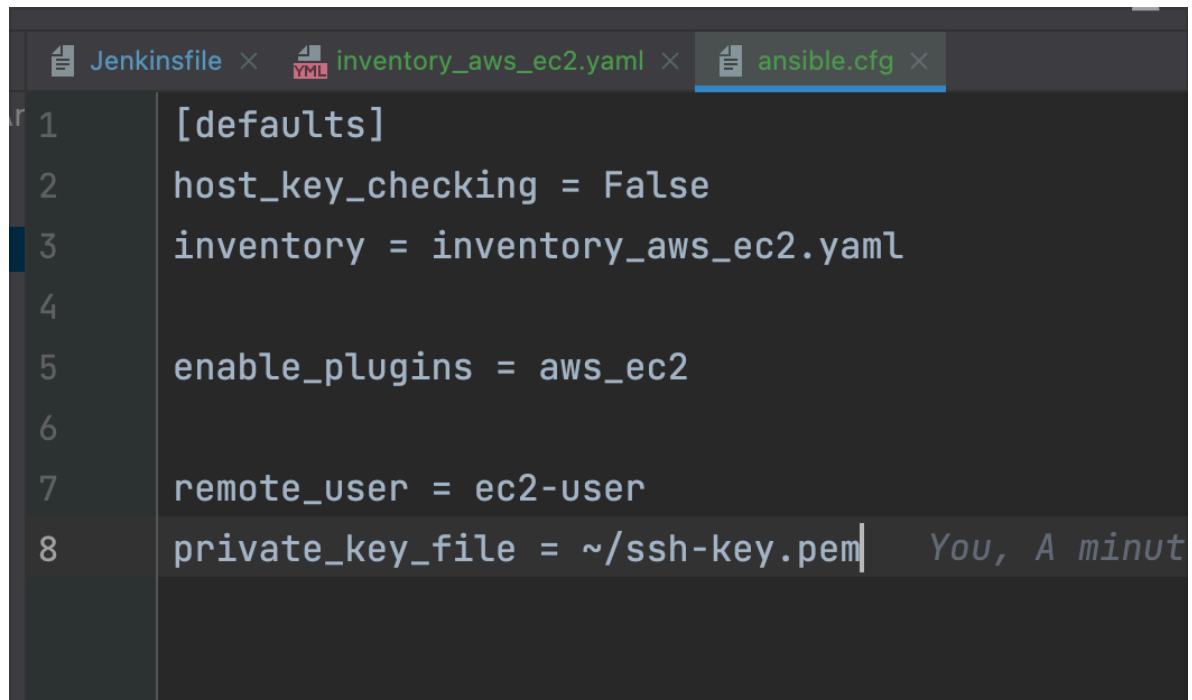


```
[defaults]
1 host_key_checking = False
2 inventory = inventory_aws_ec2.yaml
3
4
5 enable_plugins = aws_ec2
6
7 remote_user = ec2-user
8 private_key_file = ~/.ssh-key.pem
9
```

Later we will finish the pem file configuration as we still need to paste it in

jenkins?

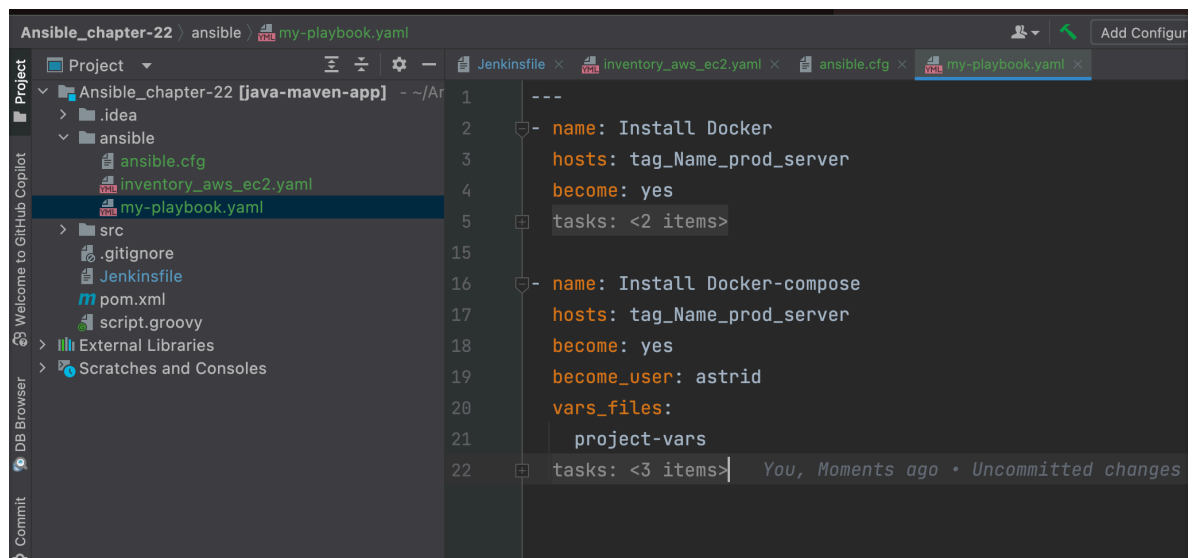
For now the ansible.cfg looks like this:



The screenshot shows an IDE with three tabs: 'Jenkinsfile', 'inventory_aws_ec2.yaml', and 'ansible.cfg'. The 'ansible.cfg' tab is active, displaying the following configuration:

```
1 [defaults]
2   host_key_checking = False
3   inventory = inventory_aws_ec2.yaml
4
5   enable_plugins = aws_ec2
6
7   remote_user = ec2-user
8   private_key_file = ~/ssh-key.pem
```

And for the p[laybook I will create a new file called 'my-playbook.yaml' and copy the docker and docker-compose part from the ansible playbook to this java-maven-app project



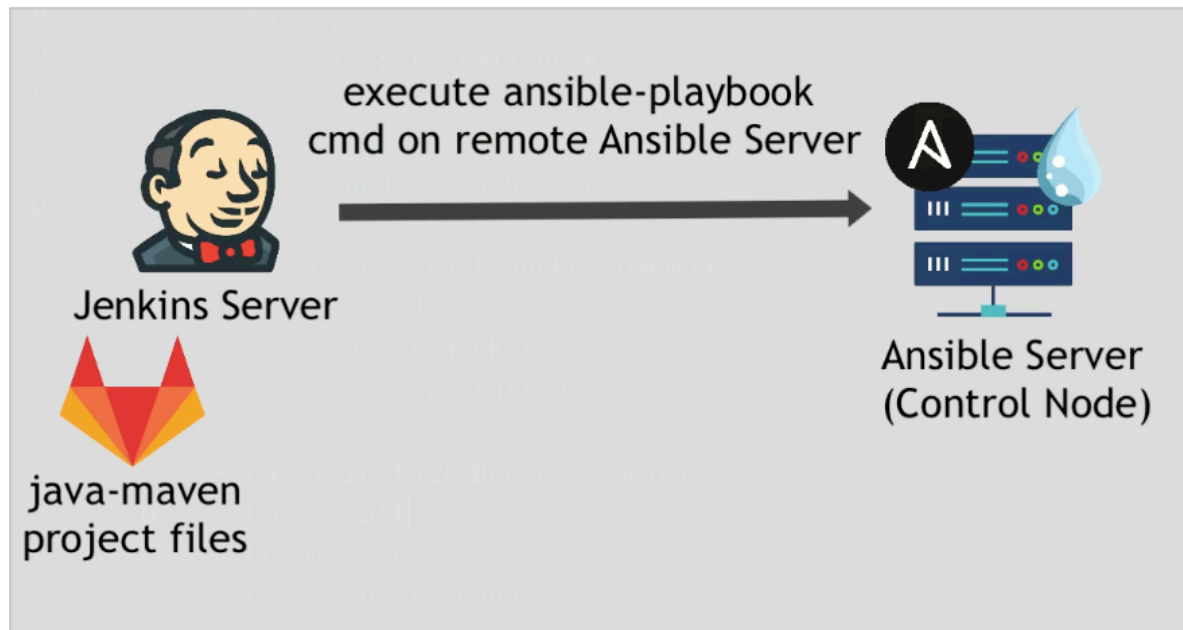
The screenshot shows an IDE with a project named 'Ansible_chapter-22 [java-maven-app]'. The 'ansible' directory contains 'ansible.cfg', 'inventory_aws_ec2.yaml', and 'my-playbook.yaml'. The 'my-playbook.yaml' file is open, showing the following tasks:

```
1 ---
2   - name: Install Docker
3     hosts: tag_Name_prod_server
4     become: yes
5     tasks: <2 items>
6
7   - name: Install Docker-compose
8     hosts: tag_Name_prod_server
9     become: yes
10    become_user: astrid
11    vars_files:
12      - project-vars
13    tasks: <3 items>
```

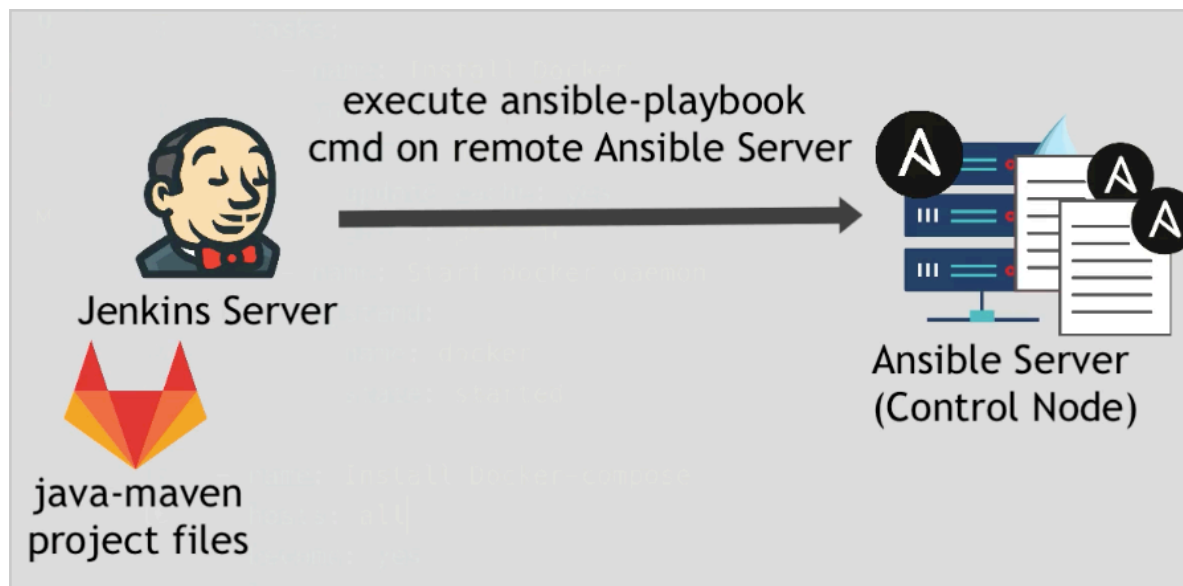
So we will use only 2 plays from the original ansible playbook.

Jenkins Server will checkout the java-maven-app repository and have all these files on the Jenkins Server when the pipeline is executed.

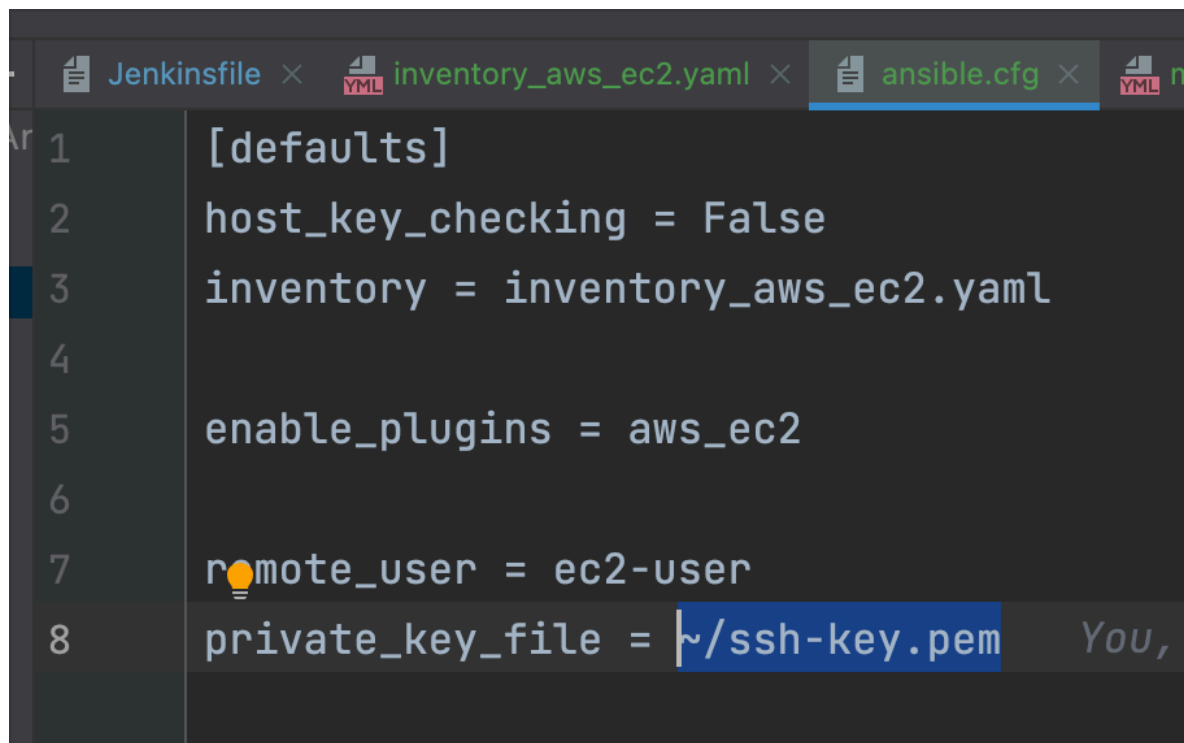
Jenkins Pipeline will trigger ansible playbook on the remote ansible server (droplet)



That means that the 3 files in 'ansible' directory need to be available on the ansible server in order for the ansible-playbook command can be executed there in the ansible server



Going back to the private key, ansible will look at the location `~/ssh-key.pem` in the ansible server and it needs to find the pem file



```
1 [defaults]
2 host_key_checking = False
3 inventory = inventory_aws_ec2.yaml
4
5 enable_plugins = aws_ec2
6
7 remote_user = ec2-user
8 private_key_file = ~/ssh-key.pem
```

So we need to copy the pem file using the pipeline into the ansible server. So this is the fourth file that needs to be available in ansible server.

How do we copy files generally from Jenkins to some remote server?

Using a plugin called ssh-agent (we did this in Module 9 - chapter 8 **Deploy to EC2 server from Jenkins Pipeline - CI/CD Part 1**)

Ssh-agent plugin allows to use and ssh key to connect to a remote server and copy all the files to that server using simple SCP -> secure copy command

Let's check that our jenkins has the ssh-agent plugin installed

The screenshot shows the Jenkins 'Manage Jenkins' interface, specifically the 'Plugins' section. On the left, there's a sidebar with 'Updates' (67), 'Available plugins', 'Installed plugins' (selected), and 'Advanced settings'. The main area has a search bar with 'ssh ag' entered. Below the search bar, the 'SSH Agent' plugin is listed with version '386.v36cc0c7582f0'. A warning message is displayed: 'Warning: The currently installed plugin version may not be safe to use. Please review the following security' followed by two links: 'OS command injection vulnerability on agents'.

And we have it!

The next thing is to configure jenkins to be able to connect to the ansible server via SSH, ansible server has an ssh key

-> my-droplet-key

Which is the one that I have stored in my laptop, my private key.

The screenshot shows the 'Authentication' section of a DigitalOcean account. There are two tabs: 'SSH Keys' (selected) and 'Password'. Below the tabs, it says 'Add an SSH Key for authentication' and 'An SSH key pair is a more secure way to connect to your Droplet'. There is a button 'Add SSH Key'. Below this, there's a list of SSH keys. The first key is 'my-droplet-key' with a checkmark and a dashed border around it. Above the list, there's a 'Select All' button and a '1/1' indicator.

And I am going to create a new global credential in Jenkins called 'ansible-server-key' and use the information from my private key

```
> ls ~/.ssh/
config          docker-server.pem  id_ed25519
```

Nana has a id_rsa file and this is the content



The newer OpenSSH format (`-----BEGIN OPENSSH PRIVATE KEY-----`, used by default since OpenSSH 7.8) isn't understood by the older SSH libraries (JSch, and forks of it) that Jenkins plugins like SSH Agent and Publish Over SSH have historically relied on.

So Nana converted the format of her rsa file to the classic ssh format that her Jenkins can handle.

```
-> ssh-keygen -p -f .ssh/id_rsa -m pem -P "" -N ""
```

`-P ""` -> current passphrase which is empty, currently the file doesn't have a passphrase

`-N ""` -> new passphrase for the converted file with the new format will be also empty ""

This converts the key into the legacy PEM (PKCS#1) container, which old JSch can parse.

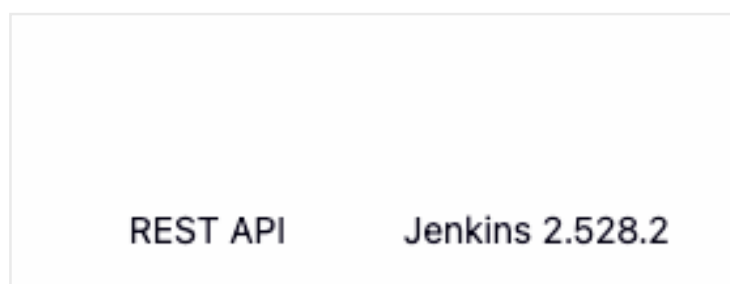
Nana created a copy and changed the name of the file with the covered format

```
nana@macbook /Users/nana
% cat .ssh/ssh_key_rsa_format
-----BEGIN RSA PRIVATE KEY-----
[blurred content]
-----END RSA PRIVATE KEY-----
nana@macbook /Users/nana
%
```

In my case I don't have and id_rsa, I have an id_ed25519 !!

That conversion doesn't work for ed25519 — PEM/PKCS#1 was designed for RSA, and ssh-keygen will refuse to export an ed25519 key that way. So even after "converting the format," you may still fail — and on top of that, older JSch-based Jenkins plugins often don't support the ed25519 algorithm at all, regardless of container format.

First check what Jenkins/plugin versions are in play — recent Jenkins with an up-to-date SSH Credentials Plugin (≥1.18-ish) and Apache MINA-based SSH support ed25519 and the new OpenSSH format just fine, so this may be a non-issue depending on setup.



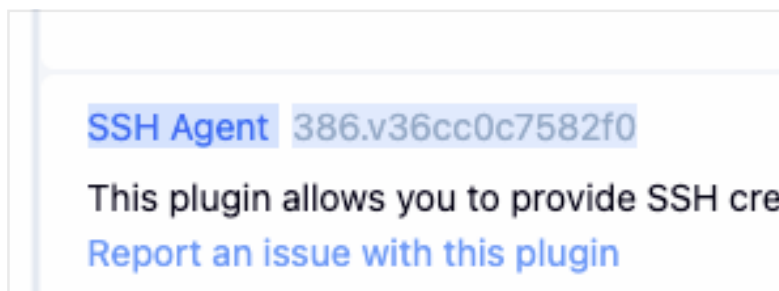
Jenkins 2.528.2 is a very recent LTS release (well into 2025/2026 — Jenkins core is currently in the 2.5xx range). That's years past when the ed25519/new-

OpenSSH-format compatibility issues were common — those problems were tied to Jenkins versions and plugin releases from around 2018–2019, before the SSH Credentials Plugin moved off pure legacy JSch parsing.

On a core this recent, any plugin you install fresh from the Jenkins Update Center will default to a current version, so you're very likely fine. That said, the plugin version is still the more precise thing to check (a Jenkins instance can technically run old plugins on a new core if they were installed long ago and never updated).

Go check: Manage Jenkins → Plugins → Installed plugins, search for whichever SSH-related plugin you're actually using (SSH Agent / SSH Credentials / Publish Over SSH / Git), and tell me the version number(s) you see. If they're recent (installed/updated in the last couple years), you should be able to use your id_ed25519 key as-is, no conversion needed.

-> SSH Agent
Version 386.v36cc0c7582f0



That version number (386.v36cc0c7582f0) follows Jenkins' newer plugin versioning scheme, where the leading number is an incrementing build counter — 386 is a recent, actively-maintained release (this scheme has only been in use since ~2022, and build 386 puts it well past that).

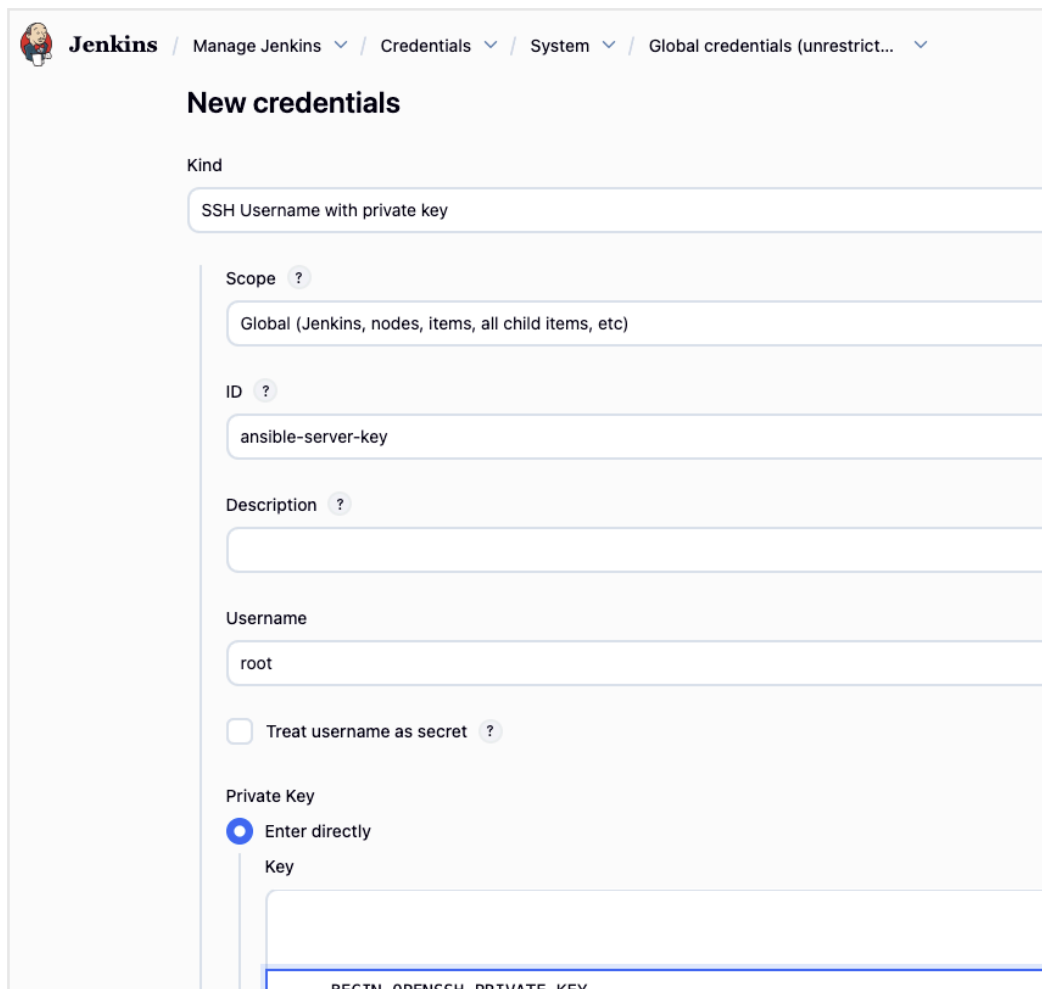
The SSH Agent plugin moved to a maintained fork of JSch (com.github.mwiede:jsch) a while back specifically to fix ed25519 and new-OpenSSH-format support — that fork is what versions in this range bundle. So on this plugin version, your id_ed25519 key in its current OpenSSH format should work fine without any conversion.

Bottom line: given Jenkins core 2.528.2 and SSH Agent plugin 386, you're on modern-enough software that you should not hit the same error your teacher did with her RSA key. If you do still get an "invalid privatekey" error when you add the credential, it'd point to something else (bad passphrase handling, wrong credential kind selected, etc.) rather than the format/algorithm issue — let me know if that happens and we can debug it properly.

So it's all good in my case 🤔

So I get the private key
-> cat ~/.ssh/id_ed25519

And use it to create the new credential



The image shows the Jenkins 'New credentials' form. The breadcrumb trail at the top is 'Jenkins / Manage Jenkins / Credentials / System / Global credentials (unrestrict...)'. The form title is 'New credentials'. The 'Kind' dropdown is set to 'SSH Username with private key'. The 'Scope' dropdown is set to 'Global (Jenkins, nodes, items, all child items, etc)'. The 'ID' field contains 'ansible-server-key'. The 'Description' field is empty. The 'Username' field contains 'root'. The checkbox 'Treat username as secret' is unchecked. Under the 'Private Key' section, the radio button 'Enter directly' is selected. Below it, the 'Key' field contains the text '-----BEGIN OPENSsh PRIVATE KEY-----'.

And it is created



The image shows a snippet of the Jenkins credentials list. It displays a single credential with the ID 'ansible-server-key', the username 'root', and the kind 'SSH Username with private key'. There is a key icon on the right side of the entry.

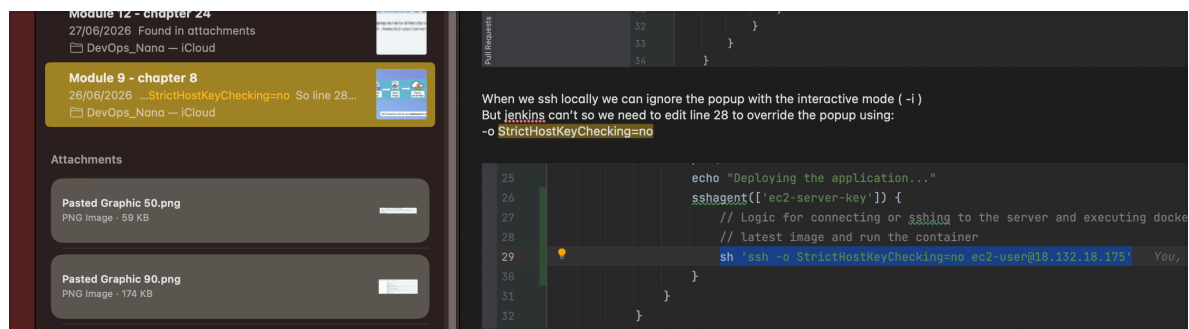
Now Jenkins will be able to ssh into the ansible server.

And we can go back to the java-maven-app and write the logic to copy the files from jenkins to ansible server using the ssh-agent and pass as an argument the credential id of the she key 'ansible-key-server'

```
Jenkinsfile x inventory_aws_ec2.yaml x ansible.cfg x my-playbook.yaml x
1 pipeline {
2   agent any
3   stages {
4     stage("copy files to ansible server") {
5       steps {
6         script {
7           echo "copying all necessary files to ansible control
8           sshagent(['ansible-server-key']) {
9             You, Moments ago • Uncommitted changes
10          }
11        }
12      }
13    }
14  }
15 }
16 }
```

And now we have a block defined by {} starting in line 8 where we can use shell or scp commands.

First let's disable the prompt question when sshing as we did in Module 9 - chapter 8



```
5 steps {
6   script {
7     echo "copying all necessary files to ansible control node"
8     sshagent(['ansible-server-key']) {
9       sh "scp -o StrictHostKeyChecking=no " You, A minute ago • Uncomm
10    }
11  }
12 }
```

Now let's copy everything from ansiabale folder in java-maven-app to the ansible server

```

6      script {
7          echo "copying all necessary files to ansible control node"
8          sshagent(['ansible-server-key']) {
9              sh "scp -o StrictHostKeyChecking=no ansible/* "
10         }

```

and the location where the files should be copy to -> /root

```

6      script {
7          echo "copying all necessary files to ansible control node"
8          sshagent(['ansible-server-key']) {
9              sh "scp -o StrictHostKeyChecking=no ansible/* 178.128.173.120:/root"
10         }
11     }

```

We also need a user -> ansible server user that has the permissions to write to / root which is the 'root' user itself.

```

6      script {
7          echo "copying all necessary files to ansible control node"
8          sshagent(['ansible-server-key']) {
9              sh "scp -o StrictHostKeyChecking=no ansible/* root@178.128.173.120:/root"
10         }
11     }

```

The user 'root' is already configured in the jenkins credentials



So jenkins will ssh into ansible server as 'root' user.

And we need to specify the user in Jenkinsfile otherwise Jenkins will use the default user which is 'jenkins' and it will fail (ansible doesn't have user 'jenkins')

```

sh "scp -o StrictHostKeyChecking=no ansible/* jenkins@138.68.94.71:/root"

```

So 3 of the 4 files will be copy at this point.

Now let's copy the ssh private key (pem rsa file I created in AWS) of the EC2 instances. Meaning we need to make the file available to jenkins in the java-maven-app first

-> ls Downloads | grep ansible

```
> ls Downloads | grep ansible
ansible-jenkins.pem
ansible.pem
natcen-ansible-use-case.html
```

It is located in my laptop's Download directory

I get the key and create a new credential in jenkins and call it 'ec2-server-key' and be careful not to include the percentage symbol in the private key:

```

Lq3EMSdsZuHrC8t7dCMQ31+Gt0Z1K69QFZuTb
7feFN5+zhvHfma0HjdptggBJiua0CEzuB7x7D1
-----BEGIN RSA PRIVATE KEY-----%
```

Jenkins / Manage Jenkins / Credentials / System / Global credentials (unrestrict...)

New credentials

Kind
SSH Username with private key

Scope ?
Global (Jenkins, nodes, items, all child items, etc)

ID ?
ec2-server-key

Description ?

Username
ec2-user

☐ Treat username as secret ?

Private Key
☒ Enter directly

Key
-----BEGIN RSA PRIVATE KEY-----

And it is created



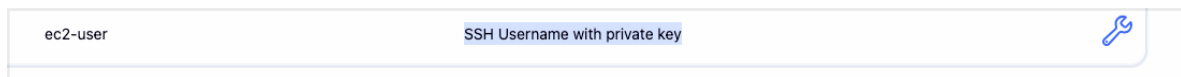
This new credentials will be used to copy the contents of its private key to the ansible server.

So now the pem file content is available in jenkins wrapped in a credential and now we can copy that to the ansible remote server.

And for that we will use 'withCredentials'

```
7      echo "copying all necessary files to ansible cont
8      sshagent(['ansible-server-key']) {
9          sh "scp -o StrictHostKeyChecking=no ansible/*
10
11      withCredentials([
12      }
13  }
14  }
15  }
16  }
```

And we pass the type of the credential



```
withCredentials([sshUserPrivateKey
```

And we pass the credentialsId as parameter to the type 'sshUserPrivateKey'

```
9      sh "scp -o StrictHostKeyChecking=no ansible/* root@178.128.173.120:/root"
10
11      withCredentials([sshUserPrivateKey(credentialsId: 'ec2-server-key')])
12  }
13  }
```

And we also pass the parameter 'keyFileVariable' which creates a temporarily file with the contents of the credential 'ec2-server-key' and we use the variable name 'keyhole' to reference it.

```

9      sh "scp -o StrictHostKeyChecking=no ansible/* root@178.128.173.120:/root"
10
11      withCredentials([sshUserPrivateKey(credentialsId: 'ec2-server-key', keyFileVariable: 'keyFile'))]
12  }

```

Then we pass the 'usernameVariable' which we can call 'user' (this 'user' will get the value 'ec2-user' from the credential)

```

8      sshagent(['ansible-server-key']) {
9          sh "scp -o StrictHostKeyChecking=no ansible/* root@178.128.173.120:/root"
10
11          withCredentials([sshUserPrivateKey(credentialsId: 'ec2-server-key', keyFileVariable: 'keyFile', usernameVariable: 'user')])
12      }
13

```

Now that we have tech the information we can now copy that information to the ansible server using shell commands and we don't need to disable the prompt question when sshing as we did in Module 9 - chapter 8 because it is already done in line 9.

So we can go ahead and define the source file

```

10
11      withCredentials([sshUserPrivateKey(credentialsId: 'ec2-server-key', keyFileVariable: 'keyFile', usernameVariable: 'user')]) {
12          sh "scp ${keyFile}" You, Moments ago • Uncommitted changes
13      }

```

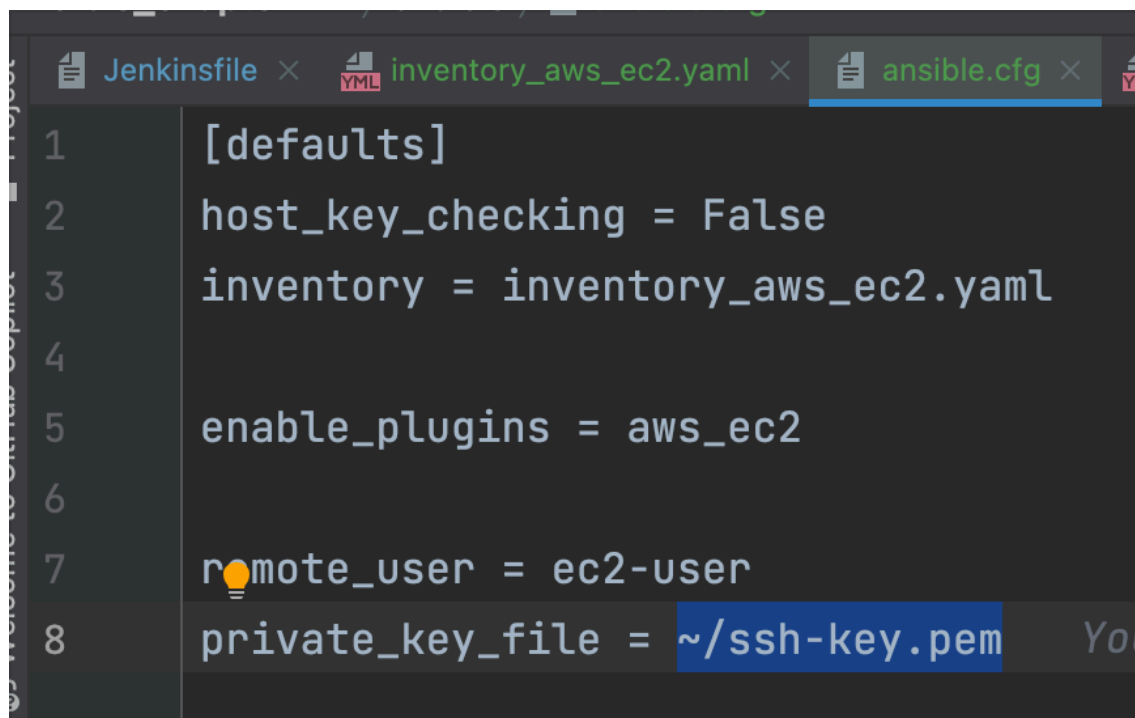
and the destination:

```

10      sh "scp -o StrictHostKeyChecking=no ansible/* root@178.128.173.120:/root"
11
12      withCredentials([sshUserPrivateKey(credentialsId: 'ec2-server-key', keyFileVariable: 'keyFile', usernameVariable: 'user')]) {
13          sh "scp ${keyFile} root@178.128.173.120:/root" You, Moments ago • Uncommitted changes
14      }
15  }

```

And we want that file to have a specify name 'ssh-key.pem' because this is the file that ansible.cfg will look for and we decided to call it like that back then



```
1 [defaults]
2 host_key_checking = False
3 inventory = inventory_aws_ec2.yaml
4
5 enable_plugins = aws_ec2
6
7 remote_user = ec2-user
8 private_key_file = ~/ssh-key.pem
```

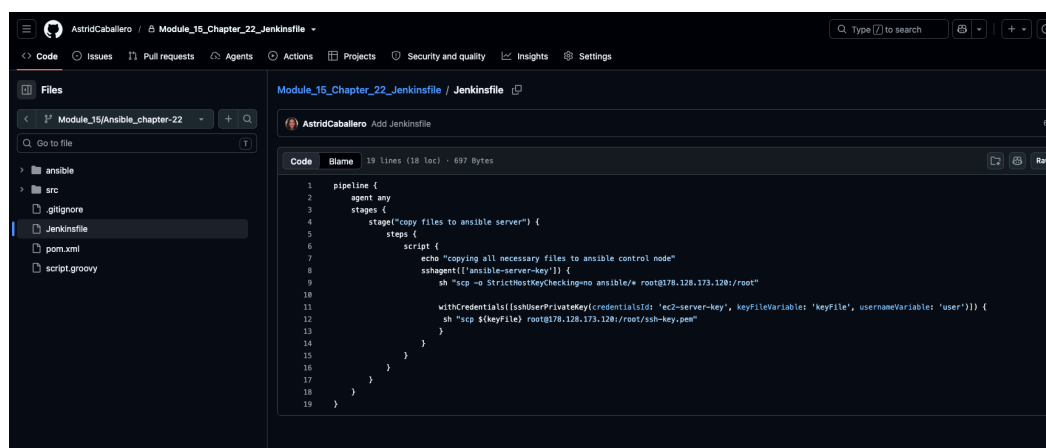
So we name it 'ssh-key.pem'



```
11 withCredentials([sshUserPrivateKey(credentialsId: 'ec2-server-key', keyFileVariable: 'keyFile', usernameVariable: 'user')]) {
12     sh "scp ${keyFile} root@178.128.173.120:/root/ssh-key.pem"
13 }
```

So keyFile (who holds the content of the private key pem file inside a Jenkins credential) will be copy to ansible server at the home directory -> /root and it will be named 'ssh-key.pem' so then ansible can use it when executing the playbook.

Let's push the branch to github



```
1 pipeline {
2   agent any
3   stages {
4     stage('copy files to ansible server') {
5       steps {
6         script {
7           echo "copying all necessary files to ansible control node"
8           sshagent(['ansible-server-key']) {
9             sh "scp -o StrictHostKeyChecking=no ansible/* root@178.128.173.120:/root/"
10           }
11           withCredentials([sshUserPrivateKey(credentialsId: 'ec2-server-key', keyFileVariable: 'keyFile', usernameVariable: 'user')]) {
12             sh "scp ${keyFile} root@178.128.173.120:/root/ssh-key.pem"
13           }
14         }
15       }
16     }
17   }
18 }
19 }
```

Now let's create a pipeline

Create Jenkins Pipeline

Configuration for a simple pipeline job like in Module 8 - chapter 8

New Item

Enter an item name

Select an item type

**Freestyle project**

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

**Pipeline**

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

Jenkins / ansible-pipeline / Configuration

Configure

- General
- Triggers
- Pipeline
- Advanced

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

git@github.com:AstridCaballero/Module_15_Chapter_22_Jenkinsfile.git

Credentials ?

AstridCaballero

Advanced

+ Add Repository

Branches to build ?

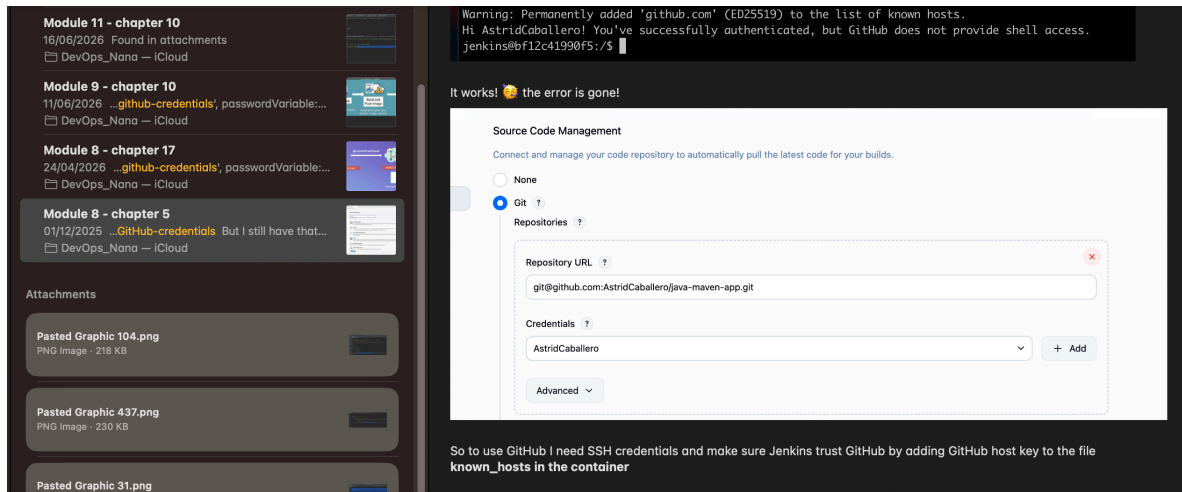
Branch Specifier (blank for 'any') ?

Module_15/Ansible_chapter-22

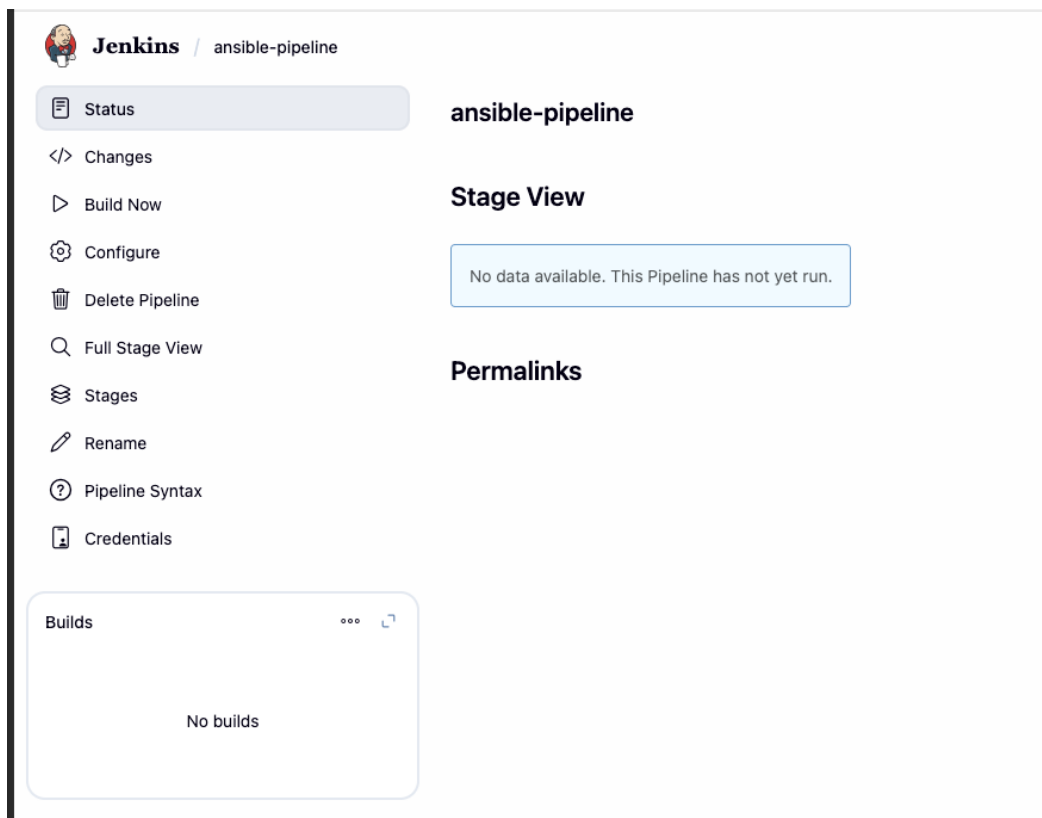
I used the credential 🙏

Credentials that should be available irrespective of domain specification to requirements matching.			
ID	Name	Kind	Description
 github-ssh-credentials	AstridCaballero	SSH Username with private key	

Because as seen in Module 8 - chapter 5, to use GitHub I need SSH credentials:



And pipeline is created:



And run the pipeline job



```
[ssh-agent] Started.
[Pipeline] {
[Pipeline] sh
+ scp -o StrictHostKeyChecking=no ansible/ansible.cfg ansible/inventory_aws_ec2.yaml ansible/my-playbook.yaml root@178.128.173.120:/root
Warning: Permanently added '178.128.173.120' (ED25519) to the list of known hosts.
[Pipeline] withCredentials
Masking supported pattern matches of $keyFile
[Pipeline] {
[Pipeline] sh
Warning: A secret was passed to "sh" using Groovy String interpolation, which is insecure.
Affected argument(s) used the following variable(s): [keyFile]
See https://jenkins.io/redirect/groovy-string-interpolation for details.
+ scp **** root@178.128.173.120:/root/ssh-key.pem
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 424600 killed;
[ssh-agent] Stopped.
[Pipeline] // sshagent
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Let's ssh into the ansible server

```
Last login: Thu Aug 15 17:03:20 2020 from 102.243.100.100
root@ansible-server:~# ls
ansible.cfg  inventory_aws_ec2.yaml  my-playbook.yaml  ssh-key.pem
root@ansible-server:~#
```

We can see that all 4 files are in the ansible server 🤔

And when ansible playbook runs and looks for the private_key_file it will find the ansible.cfg and it will go to ~/ssh-key.pem and it will find it

Below screenshot of ansible.cfg content:

```

root@ansible-server:~# cat ~/ansible.cfg
[defaults]
host_key_checking = False
inventory = inventory_aws_ec2.yaml

enable_plugins = aws_ec2

remote_user = ec2-user
private_key_file = ~/ssh-key.pemroot@ansible-server:~# █

```

And it will be able to fetch the AWS inventory using the dynamic inventory plugin.

The pipeline is giving us a warning sign regarding exposing a private key file:

The screenshot shows the Jenkins interface for a pipeline run. On the left is a sidebar with navigation links: Status, Changes, Console Output, Edit Build Information, Delete build '#1', Timings, Git Build Data, Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, and Workspaces. The main area displays the run details for '#1 (Aug 14, 2026, 9:41:22 AM)'. It indicates the run was started by user 'astrid caballero' and shows timing information: 23 ms waiting, 7.8 sec build duration, and 7.8 sec total from scheduled to completion. Below this, it shows the Git revision and repository. A yellow warning icon is present, with a message: 'The following steps that have been detected may have insecure interpolation of sensitive variables (click here for an explanation):'. The message lists one step: 'sh: [keyFile]'. At the bottom, it states 'No changes.'

In the console output we can see the warning as well:

```

[Pipeline] sh
Warning: A secret was passed to "sh" using Groovy String interpolation, which is insecure.
Affected argument(s) used the following variable(s): [keyFile]
See https://jenkins.io/redirect/groovy-string-interpolation for details.

```

The groovy interpreter exposes the content of `${keyFile}` 📌 on the command line

```

withCredentials([sshKey]
sh "scp ${keyFile}

```

So it is going to be in the command line history -> insecure !!

Fix:

Use a different syntax:

-> single quote

-> remove {}

```
11 withCredentials([sshUserPrivateKey(credentialsId: 'ec2-serv  
12 sh 'scp $keyFile root@178.128.173.120:/root/ssh-key.pem'
```

Now the secret won't be expose to the command line.

So I push the changes to GitHub and run a new pipeline job





#2 (Aug 14, 2026, 10:04:53 AM)



Started by user [astrid caballero](#)



This run spent:

- 17 ms waiting;
- 6.1 sec build duration;
- 6.1 sec total from scheduled to completion.



Revision: 92b0b315254165feb4701f431a7d2b15494dff3

Repository: [git@github.com:AstridCaballero/Module_15_Chapter_22_Jenkinsfile.git](#)

- [refs/remotes/origin/Module_15/Ansible_chapter-22](#)



Changes

1. Fix exposing secret to command line ([details](#) / [githubweb](#))

```
$ ssh-agent
SSH_AUTH_SOCK=/tmp/ssh-xx54Zji0xqB7/agent.424837
SSH_AGENT_PID=424840
Running ssh-add (command line suppressed)
Identity added: /var/jenkins_home/workspace/ansible-pipeline/tmp/private_key_13862826643761788644.key (astridcaballero@gmail.com)
[ssh-agent] Started.
[Pipeline] {
[Pipeline] sh
+ scp -o StrictHostKeyChecking=no ansible/ansible.cfg ansible/inventory_aws_ec2.yaml ansible/my-playbook.yaml root@178.128.173.120:/root
[Pipeline] withCredentials
Masking supported pattern matches of $keyFile
[Pipeline] {
[Pipeline] sh
+ scp ***** root@178.128.173.120:/root/ssh-key.pem
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 424840 killed;
[ssh-agent] Stopped.
[Pipeline] // sshagent
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

No warning 🤔

Jenkinsfile: Execute Ansible Playbook from Jenkins

Now that we have copied the files to the ansible server we can execute the playbook from Jenkins Server by triggering it on the ansible remote server.

So we add a new stage to Jenkinsfile -> 'execute ansible playbook'

```

17     }
18     stage ("execute ansible playbook") {
19         steps {
20             script {
21                 echo "calling ansible playbook to configure EC2 instances"
22                 You, Moments ago • Uncommitted changes
23             }
24         }
25     }

```

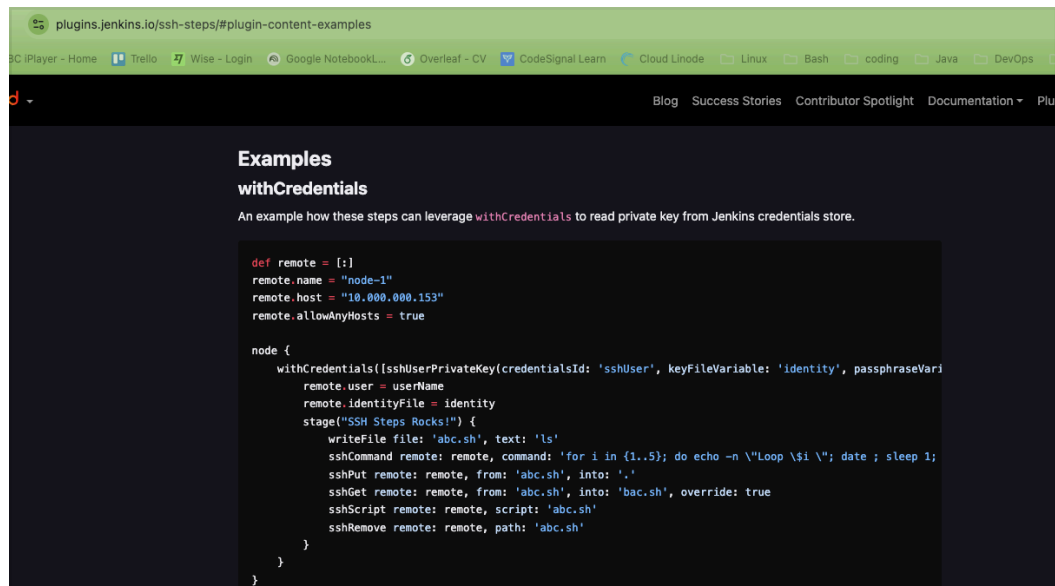
And now we need another plugin 'ssh pipeline steps' that will enable us to execute command line commands on a remote server

The screenshot shows the Jenkins 'Manage Jenkins' interface, specifically the 'Plugins' section. On the left, there's a sidebar with 'Updates' (67), 'Available plugins', 'Installed plugins', and 'Advanced settings'. The main area has a search bar with 'ssh pip' and a table of plugins. The 'SSH Pipeline Steps' plugin (version 2.0.92.vb_a_0583935f9b_) is listed with an 'Install' checkbox. Below the name, it says 'Jenkins pipeline steps which provides SSH facilities such as command execution or file transfer'.

The screenshot shows the 'Download progress' page in Jenkins. The left sidebar is the same as the previous screenshot, but 'Download progress' is selected. The main area shows the progress of downloading the 'SSH Pipeline Steps' plugin. It lists 'Preparation' (Checking internet connectivity, Checking update center connectivity, Success), 'JSch dependency' (Success), 'SSH Pipeline Steps' (Success), and 'Loading plugin extensions' (Success). At the bottom, there's a link to 'Go back to the top page' and a checkbox to 'Restart Jenkins when installation is complete and no jobs are running'.

Here is the documentation with examples

<https://plugins.jenkins.io/ssh-steps>



We will execute a simple sshCommand

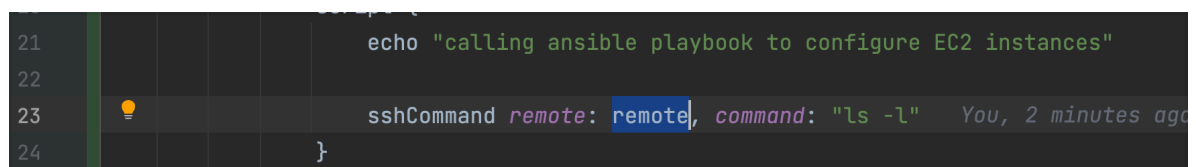


And we will create an object called 'remote' which holds multiple attributes referencing the remote server.

We pass two parameters:

- remote
- command

So we are executing a command on a remote server and we have to pass in an object for the remote server including the host name, the IP address, the user, private key, etc



```

- sshCommand - a pipeline step provided by the SSH Steps plugin. It opens an SSH connection to a remote host, runs a single command, and returns the output.
- remote: remote - the named parameter remote is set to a variable also called remote (Groovy allows this - the value on the right is a variable in scope, the key on the left is the parameter name). That remote variable is typically a Map defined earlier in the Jenkinsfile, e.g.:
def remote = [:]
remote.name = 'my-server'
remote.host = '10.0.0.5'
remote.user = 'deploy'
remote.identityFile = '/path/to/key' // or remote.password = '...'
remote.allowAnyHosts = true
- This map tells the plugin where to connect and how to authenticate.
- command: "ls -l" - the actual shell command to execute on that remote host once connected - here, listing the contents of the current remote directory in long format.

So in plain English: "Connect via SSH to the server described by the remote map, run ls -l there, and return the result."

```

Now we define the variable 'remote' -> a single grove object with multiple parameters -> key-value pair

```

echo "calling ansible playbook to configure EC2 instances"
def remote: [:]    You, 4 minutes ago • Uncommitted changes
sshCommand remote: remote, command: "ls -l"

```

So the remote is empty at the moment and now let's set some values:

- Name of the remote, let's call it 'ansible-server'

```

echo "calling ansible playbook to configure EC2 instances"
def remote: [:]
remote.name = "ansible-server"    You, A minute ago • Uncommitted changes
sshCommand remote: remote, command: "ls -l"

```

- IP address of the remote server

```

echo "calling ansible playbook to configure EC2 instances"
def remote: [:]
remote.name = "ansible-server"
remote.host = "178.128.173.120"    You, Moments ago • Uncommitted changes
sshCommand remote: remote, command: "ls -l"

```

To avoid prompt questions to be set interactively, i.e. like when we ssh manually into a server and we get the question and we have to input 'y'

We set allowAnyHosts = true

```
remote.name = "ansible-server"
remote.host = "178.128.173.120"
remote.allowAnyHosts = true
```

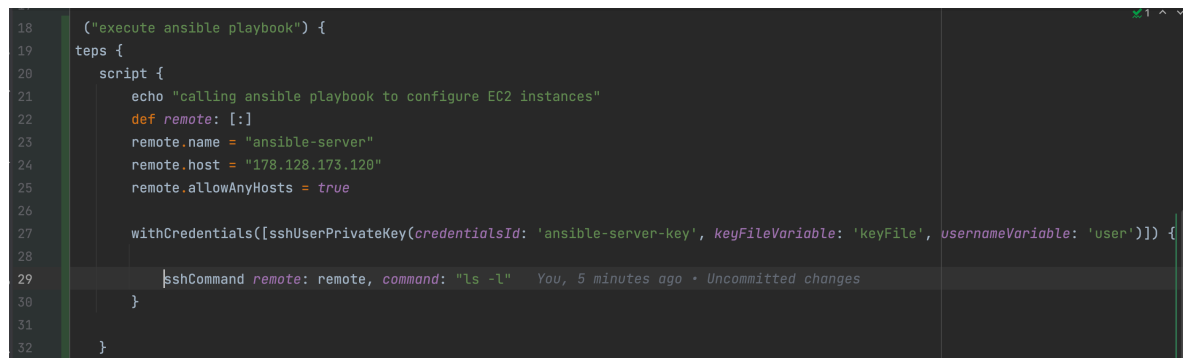
And when connecting to a remote server we need the user name and the private key of the remote server -> ansible server
And we will use the ansible credential that we created in jenkins



The screenshot shows the Jenkins configuration page for a credential named 'ansible-server-key'. The breadcrumb trail at the top is 'System (global) ansible-server-key root'. The main content area shows a code editor with the following Ansible configuration:

```
22 def remote: [:]
23   remote.name = "ansible-server"
24   remote.host = "178.128.173.120"
25   remote.allowAnyHosts = true
26
27   withCredentials([sshUserPrivateKey(credentialsId: 'ansible-server-key', keyFileVariable: 'keyFile', usernameVariable: 'user')]) {
28
29   }
30
31
32   sshCommand remote: remote, command: "ls -l"
33
```

And inside the credentials block we can execute the sshCommand



The screenshot shows the Jenkins configuration page for a credential named 'ansible-server-key'. The breadcrumb trail at the top is 'System (global) ansible-server-key root'. The main content area shows a code editor with the following Ansible configuration:

```
18 ("execute ansible playbook") {
19   steps {
20     script {
21       echo "calling ansible playbook to configure EC2 instances"
22       def remote: [:]
23         remote.name = "ansible-server"
24         remote.host = "178.128.173.120"
25         remote.allowAnyHosts = true
26
27         withCredentials([sshUserPrivateKey(credentialsId: 'ansible-server-key', keyFileVariable: 'keyFile', usernameVariable: 'user')]) {
28
29           sshCommand remote: remote, command: "ls -l"
30         }
31       }
32     }
33
```

And now we set the values of the credentials to the remote object:

```

20 script {
21     echo "calling ansible playbook to configure EC2 instances"
22     def remote: [:]
23         remote.name = "ansible-server"
24         remote.host = "178.128.173.120"
25         remote.allowAnyHosts = true
26
27         withCredentials([sshUserPrivateKey(credentialsId: 'ansible-server-key', keyFileVariable: 'keyFile', usernameVariable: 'user')]) {
28             remote.user = user
29             remote.identityFile = keyFile
30             sshCommand remote: remote, command: "ls -l"
31         }
32     }
33 }

```

We can run multiple commands:

```

30 sshCommand remote: remote, command: "ls -l; pwd"

```

Or

```

sshCommand remote: remote, command: "ls -l"
sshCommand remote: remote, command: "ls -l"

```

Let's test the configuration we have so far using command
-> ls -l

So I push the changes to GitHub repo.

And build the pipeline again and it failed because I had a typo ':' instead of '='

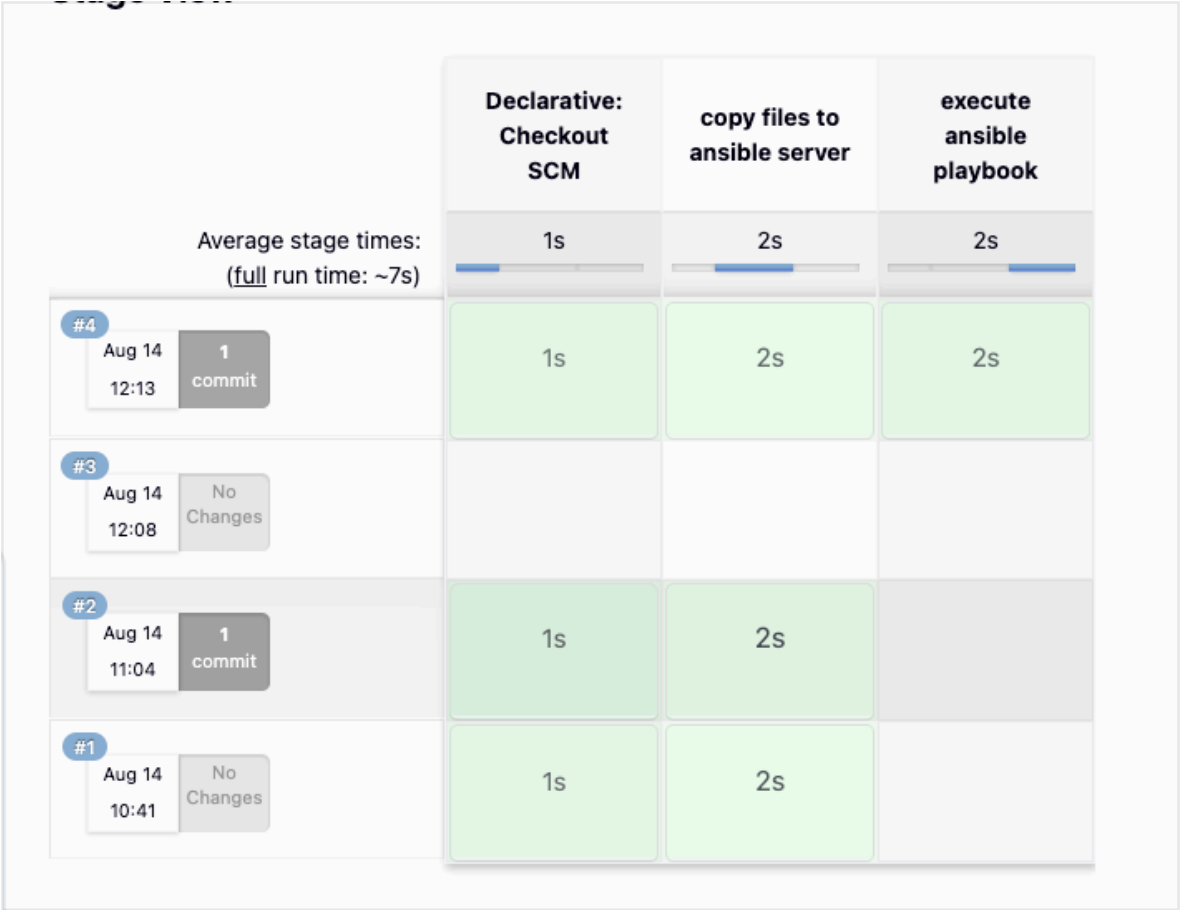
```

21 echo "calling ansible playbook to c
22 def remote = [:]
23     .name = "ansible-server"
24     def remote: [:]
25     remote.allowAnyHosts = true

```

So I push the changes to GitHub repo.

And build the pipeline again



```

calling ansible playbook to configure EC2 instances
[Pipeline] withCredentials
Masking supported pattern matches of $keyFile
[Pipeline] {
[Pipeline] sshCommand
Executing command on ansible-server[178.128.173.120]: ls -l sudo: false
total 16
-rw-r--r-- 1 root root 154 Aug 14 11:13 ansible.cfg
-rw-r--r-- 1 root root 148 Aug 14 11:13 inventory_aws_ec2.yaml
-rw-r--r-- 1 root root 905 Aug 14 11:13 my-playbook.yaml
-r----- 1 root root 1676 Aug 14 11:13 ssh-key.pem
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

It worked 🤔 we can see the list of files in the ansible remote server and their permissions from the execution of 'ls -l'

So now that we tested that it works, now let's execute the 'ansible-playbook' command in the remote server

```

20 script {
21     echo "calling ansible playbook to configure EC2 instances"
22     def remote = [:]
23     remote.name = "ansible-server"
24     remote.host = "178.128.173.120"
25     remote.allowAnyHosts = true
26
27     withCredentials([sshUserPrivateKey(credentialsId: 'ansible-server-key', keyFileVariable: 'keyFile', usernameVariable: 'user')]) {
28         remote.user = user
29         remote.identityFile = keyFile
30         sshCommand remote: remote, command: "ansible-playbook my-playbook.yaml"
31     }
32 }
33

```

Ansible server has everything it needs to run the my-playbook.yaml so all we need to do is to execute it.

So I push the changes to GitHub repo.

And build the pipeline again

✅ ansible-pipeline

Stage View

		Declarative: Checkout SCM	copy files to ansible server	execute ansible playbook
Average stage times: (full run time: ~8s)		1s	2s	2s
#5	Aug 14 12:20 1 commit	1s	2s	3s
#4	Aug 14 12:13 1 commit	1s	2s	2s
#3	Aug 14 12:08 No Changes			
#2	Aug 14 11:04 1 commit	1s	2s	
#1	Aug 14 10:41 No Changes	1s	2s	

```

[Pipeline] echo
calling ansible playbook to configure EC2 instances
[Pipeline] withCredentials
Masking supported pattern matches of $keyFile
[Pipeline] {
[Pipeline] sshCommand
Executing command on ansible-server[178.128.173.120]: ansible-playbook my-playbook.yaml sudo: false

PLAY [Install Docker] *****
skipping: no hosts matched

PLAY [Install Docker-compose] *****
[WARNING]: Could not match supplied host pattern, ignoring:
tag_Name_prod_server
skipping: no hosts matched

PLAY RECAP *****

[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

It sort of worked but the playbook failed.

```

PLAY [Install Docker] *****
skipping: no hosts matched

PLAY [Install Docker-compose] *****
[WARNING]: Could not match supplied host pattern, ignoring:
tag_Name_prod_server
skipping: no hosts matched

PLAY RECAP *****

```

So I checked my-playbook and the hosts is wrong

```

1  ---
2  - name: Install Docker
3    hosts: tag_Name_prod_server Pardo, Yesterday
4    become: yes
5    tasks:
6      - name: Make sure Docker is installed
7        yum:
8          name: docker
9          update_cache: yes
10         state: present
11      - name: Ensure Docker is running
12        systemd:
13          name: docker
14          state: started
15
16  - name: Install Docker-compose
17    hosts: tag_Name_prod_server
18    become: yes
19    become_user: astrid
20    vars_files:
21      project-vars

```

It needs to be hosts: all

```

1  ---
2  - name: Install Docker
3    hosts: all
4    become: yes
5    tasks:
6      - name: Make sure Docker is installed
7        yum:
8          name: docker
9          update_cache: yes
10         state: present
11      - name: Ensure Docker is running
12        systemd:
13          name: docker
14          state: started
15
16 - name: Install Docker-compose
17   hosts: all

```

And I have another inconsistency, I am not creating a new user so the task 'Install Docker-compose' doesn't need:
 become: yes
 become_user: astrid

And also doesn't need
 vars_files:
 project-vars

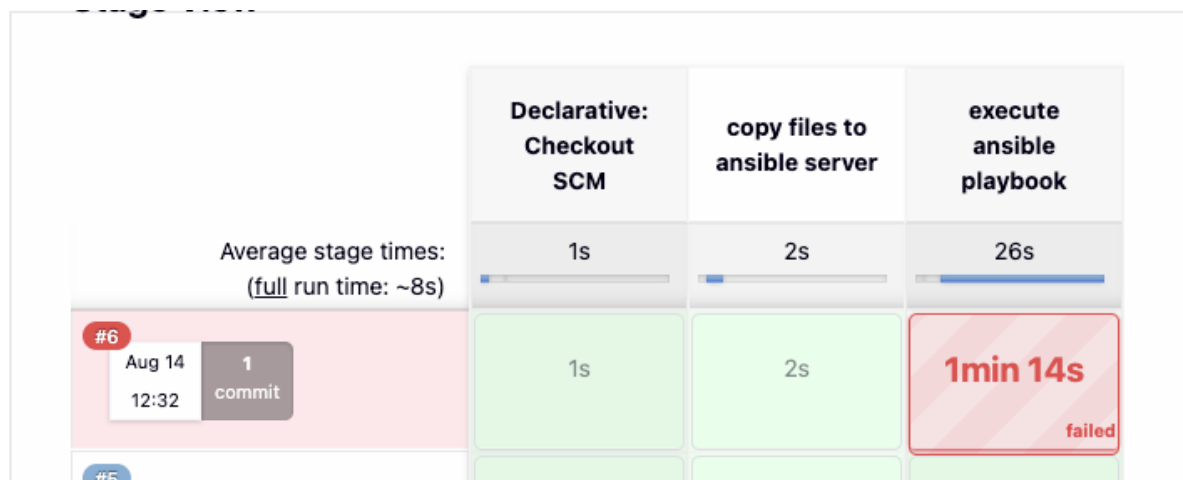
```

16  - name: Install Docker-compose
17      hosts: all
18      become: yes    Pardo, Yesterday • Add Jenkinsfile
19      become_user: astrid
20      vars_files:
21          - project-vars
22      tasks:
23          - name: Create docker-compose directory
24            file:
25              path: ~/.docker/cli-plugins
26              state: directory
27          - name: get architecture of remote machine
28            shell: uname -m
29            register: remote_arch
30          - name: Install docker-compose
31            get_url:
32              url: https://github.com/docker/compose/releases
33              dest: ~/.docker/cli-plugins/docker-compose
34              mode: +x

```

I remove those lines, I push the changes to GitHub repo.

And build the pipeline again and it failed



An ansible task failed

```

TASK [get architecture of remote machine] *****
changed: [ec2-18-170-46-239.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-50-173.eu-west-2.compute.amazonaws.com]

TASK [Install docker-compose] *****
fatal: [ec2-18-170-46-239.eu-west-2.compute.amazonaws.com]: FAILED! => {"msg": "The task includes an option with an undefined variable. The error was: 'docker_compose_version' is undefined. 'docker_compose_version' is undefined\n\nThe error appears to be in '/root/my-playbook.yml': line 26, column 7, but may\nbe elsewhere in the file depending on the exact syntax problem.\n\nThe offending line appears to be:\n\n    register: remote_arch\n    - name: Install docker-compose\n      ^ here\n"}
fatal: [ec2-18-170-50-173.eu-west-2.compute.amazonaws.com]: FAILED! => {"msg": "The task includes an option with an undefined variable. The error was: 'docker_compose_version' is undefined. 'docker_compose_version' is undefined\n\nThe error appears to be in '/root/my-playbook.yml': line 26, column 7, but may\nbe elsewhere in the file depending on the exact syntax problem.\n\nThe offending line appears to be:\n\n    register: remote_arch\n    - name: Install docker-compose\n      ^ here\n"}

PLAY RECAP *****
ec2-18-170-46-239.eu-west-2.compute.amazonaws.com : ok=6    changed=4    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
ec2-18-170-50-173.eu-west-2.compute.amazonaws.com : ok=6    changed=4    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0

```

Because I remove the
vars_files:
project-vars

Which is holding the value of variable 'docker_compose_version'

```

27  get_url:
28  url: https://github.com/docker/compose/releases/download/v{{docker_compose_version}}/docker-compose-linux-{{remote_arch.stdout}}
29  dest: ~/.docker/cli-plugins/docker-compose
30  mode: +x

```

So I need to fix that, I will hardcode the version

```

16  name: Install Docker-compose
17  hosts: all
18  tasks:
19  - name: Create docker-compose directory
20    file:
21      path: ~/.docker/cli-plugins
22      state: directory
23  - name: get architecture of remote machine
24    shell: uname -m
25    register: remote_arch
26  - name: Install docker-compose
27    get_url:
28      url: https://github.com/docker/compose/releases/download/v5.4.0/docker-compose-linux-{{remote_arch.stdout}}
29      dest: ~/.docker/cli-plugins/docker-compose
30      mode: +x

```

I push the changes to GitHub repo.

And build the pipeline again

✅ ansible-pipeline

Stage View

		Declarative: Checkout SCM	copy files to ansible server	execute ansible playbook
Average stage times: (full run time: ~10s)		1s	2s	23s
#7	Aug 14 12:43 1 commit	1s	2s	11s
#6	Aug 14 12:32 1 commit	1s	2s	1min 14s failed
#5	Aug 14 12:20 1 commit	1s	2s	3s
#4	Aug 14 12:13 1 commit	1s	2s	2s
#3	Aug 14 No			

```
[Pipeline] sshCommand
Executing command on ansible-server[178.128.173.120]: ansible-playbook my-playbook.yaml sudo: false

PLAY [Install Docker] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host ec2-18-170-46-239.eu-west-2.compute.amazonaws.com is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference\_appendices/interpreter\_discovery.html for more information.
[WARNING]: Platform linux on host ec2-18-170-50-173.eu-west-2.compute.amazonaws.com is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference\_appendices/interpreter\_discovery.html for more information.
ok: [ec2-18-170-46-239.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-170-50-173.eu-west-2.compute.amazonaws.com]

TASK [Make sure Docker is installed] *****
ok: [ec2-18-170-46-239.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-170-50-173.eu-west-2.compute.amazonaws.com]

TASK [Ensure Docker is running] *****
ok: [ec2-18-170-46-239.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-170-50-173.eu-west-2.compute.amazonaws.com]

PLAY [Install Docker-compose] *****

TASK [Gathering Facts] *****
ok: [ec2-18-170-46-239.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-170-50-173.eu-west-2.compute.amazonaws.com]

TASK [Create docker-compose directory] *****
ok: [ec2-18-170-46-239.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-170-50-173.eu-west-2.compute.amazonaws.com]
```

```
TASK [get architecture of remote machine] *****
changed: [ec2-18-170-46-239.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-50-173.eu-west-2.compute.amazonaws.com]

TASK [Install docker-compose] *****
changed: [ec2-18-170-46-239.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-50-173.eu-west-2.compute.amazonaws.com]

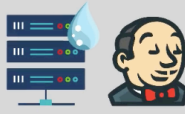
PLAY RECAP *****
ec2-18-170-46-239.eu-west-2.compute.amazonaws.com : ok=7    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ec2-18-170-50-173.eu-west-2.compute.amazonaws.com : ok=7    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

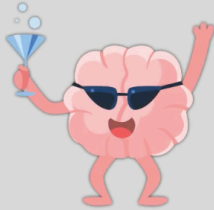
And it worked 🥳✅❤️

The dynamically fetched Ec2 servers were configured and now have docker and docker-compose installed.

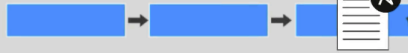
Project: Ansible Integration in Jenkins



Jenkins Server



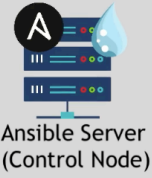
- 1) Create a DO server for Jenkins



- 2) Create dedicated server for Ansible

- 3) Install Ansible on that server

- 4) Execute Ansible Playbook from Jenkins Pipeline to configure 2 EC2 Instances



Ansible Server
(Control Node)

